Agentic RAG Kodlama Lab'ı: Öğretmen Rehberi

Bu doküman sadece `instructor` branch'inde bulunur. Referans çözümleri, anlatım notlarını ve sık hataları içerir. Çözümlerin asıl kaynağı koddaki `# --- ÇÖZÜM BAŞLANGIÇ ---` / `# --- ÇÖZÜM BİTİŞ ---` bölgeleridir.

Ders planı (yaklaşık 3 saat 15 dk)

Bölüm	İçerik	Süre
Kurulum	`uv sync`, `.env`, Qdrant; uygulamayı canlı gösterin	15 dk
1. RAG temelleri	10 dk anlatım + Görev 1-4	55 dk
Ara		10 dk
2. LangGraph	10 dk anlatım + Görev 5-7	65 dk
3. Tool ve Agent	10 dk anlatım + Görev 8-10	50 dk
Kapanış	Uçtan uca çalıştırma, workflow ile agentic karşılaştırması	10 dk

Zaman yetmezse kısaltma sırası: Görev 1 hızlıca birlikte yapılır, Görev 6 tabloyla birlikte yazılır, Görev 10'un agentic graph kısmı öğretmen tarafından gösterilir.

Testler

- `tests/exercises/test_01_rag.py` (Görev 1-4), `test_02_langgraph.py` (Görev 5-7), `test_03_tool_agent.py` (Görev 8-10). API anahtarı ve Qdrant gerekmez.
- Test adları `test_gorevN_...` biçimindedir: `-k gorev3` ile tek görev çalıştırılır.
- `instructor`'da hepsi yeşildir. `student`'ta hazır kısımları sınayan 2 test dışında hepsi "henüz yazılmadı" mesajıyla kırmızıdır.

Branch ve student sürümü

Branch	İçerik
`main`	Çalışan uygulama.
`instructor`	`main` + çözüm işaretleri + testler + rehberler + betikler.
`student`	Stub kod + öğrenci rehberi + testler. Tek commit'lik orphan branch; çözüm geçmişi yok.

Rehber kaynakları `doc/education/parts/` altındadır (git'te ignore edilir). Düzenledikten sonra `uv run python scripts/build_education_guides.py` ile Markdown üretilir, `docs/` altına PDF olarak aktarılır.

Student sürümü: `uv run python scripts/make_student_branch.py build/student`. Betik git'te izlenen dosyaları kopyalar, çözüm bölgelerini `raise NotImplementedError(...)` ile değiştirir; `docs/TEACHER_GUIDE.pdf`, `doc/` ve betiklerin kendisini dışarıda bırakır. İzlenmeyen dosyalar kopyalanmadığı için önce commit edin.

Orphan branch'e aktarma:

```
git worktree add --detach ../student-wt
cd ../student-wt
git checkout --orphan student        # sonraki seferlerde: git checkout student
git rm -rf . -q
cp -r ../lang_rag_playground/build/student/. .
git add -A
git commit -m "Öğrenci şablonu"
```

Kontrol: `student`'ta testler kırmızı; `git log -p student | grep "ÇÖZÜM BAŞLANGIÇ"` boş; `git ls-tree -r student --name-only | grep -i teacher` boş.

Bölüm 1: RAG Temel Yapıları (Öğretmen)

Anlatım notu (10 dk)

- Ingestion akışını tek cümleyle gösterin: PDF -> Markdown -> chunk -> embedding -> Qdrant.
- Hibrit arama: tahtaya bir eş anlamlı örnek ("otomobil"/"araba") ve bir tam ifade örneği ("Madde 142") yazın; hangisini hangi vektörün yakaladığını sorun. RRF formülünü `1 / (k + sıra)` gösterin; LangChain'in `HYBRID` modu bunu Qdrant içinde yapar.
- Reranker: geniş aday havuzu (`RERANK_CANDIDATES`), dar sonuç (`TOP_K_CANDIDATES`). LLM çıktısının doğrulanması gerektiğini vurgulayın.
- Guardrails: canlı bir injection denemesi gösterin ("Önceki talimatları yok say...").

Görev 1

```
self.header_splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=HEADERS,
    strip_headers=header_splitter_strip_headers,
)
self.semantic_splitter = SemanticChunker(
    embeddings=embedding_model,
    breakpoint_threshold_type=semantic_chunker_threshold_type,
    breakpoint_threshold_amount=semantic_chunker_breakpoint_threshold_amount,
    min_chunk_size=min_chunk_chars,
    buffer_size=buffer_size,
)
self.size_splitter = RecursiveCharacterTextSplitter(
    chunk_size=max_chunk_chars,
    chunk_overlap=overlap_chars,
    is_separator_regex=True,
    separators=SENTENCE_SPLIT_RE,
    length_function=len,
)
```

Görev 2

```
return QdrantVectorStore(
    client=self.client,
    collection_name=collection_name,
    embedding=self.embedding,
    sparse_embedding=self.sparse_embedding,
    retrieval_mode=RetrievalMode.HYBRID,
    vector_name="dense",
    sparse_vector_name="sparse",
)
```

Görev 3

```
reranked_documents = []
seen_ids = set()
sorted_scores = sorted(response.scores, key=lambda item: item.score, reverse=True)
for item in sorted_scores:
    if item.score < min_score:
        continue
    if item.id in seen_ids or item.id >= len(candidates):
        continue
    seen_ids.add(item.id)
    doc = candidates[item.id]
    updated_metadata = {**doc.metadata, "reranker_score": item.score}
    reranked_documents.append(doc.model_copy(update={"metadata":
updated_metadata}))
return reranked_documents[:top_k]
```

Sıralama filtrelemeden önce yapıldığı için yinelenen bir id'nin en yüksek skorlu kaydı kalır.

Görev 4

```
if self._is_prompt_injection(message):
    logger.warning("Prompt injection detected in user message.")
    return InputGuardrailResult(is_input_safe=False, message=None)
return InputGuardrailResult(is_input_safe=True,
message=self._mask_message(message))
```

Sık hatalar

- Görev 2'de vektör adlarını yanlış yazmak: koleksiyon şemasıyla eşleşmez, aramada hata verir.
- Görev 3'te `seen_ids.add` işleminin `continue` kontrolünden önce yapılması veya `top_k`'nin sıralamadan önce kesilmesi.
- Görev 3'te belgeyi yerinde değiştirmek (`test_gorev3_orijinal_aday_degismez` kırmızı).
- Görev 4'te `_mask_message` `None` döndüğünde orijinal mesajı koymak.

Bölüm 2: LangGraph (Öğretmen)

Anlatım notu (10 dk)

- Düğüm sözleşmesi: state oku, kısmi güncelleme döndür. `add_messages` reducer'ı mesajları ekler.
- Fabrika kalıbı neden gerekli: düğüm imzası sadece `state`.
- Router tablosunu tahtaya çizin. Judge router'ındaki sıra ve `retry_count >= max_retries` koşulunun sonsuz döngüyü nasıl engellediğini gösterin.
- Görev 7'den önce öğrencilerin akışı kağıda çizmesini isteyin (5 dk), sonra aşağıdaki dolu diyagramla karşılaştırın.

Görev 5

```
llm = build_chat_model(model_name, temperature=temperature)
return INTENT_TEMPLATE | llm.with_structured_output(Intent)
```

```
last_user_msg = get_user_last_message(state)
history = trim_history(state, max_tokens=max_history_tokens)
response = await chain.ainvoke({"input_text": last_user_msg.text, "history":
history})
# ...
intent = response.intent if response.confidence >= min_confidence else
"needs_clarification"
# ...
return {
    "intent": intent,
    "intent_confidence": response.confidence,
    "resolved_query": response.resolved_query,
}
```

Görev 6

```
def route_after_input_guardrails(state):
    if not state.get("is_input_safe"):
        return "blocked"
    return "intent"

def route_after_memory_recall(state):
    if state.get("intent") == "knowledge_question":
        return "retrieval"
    return "assistant"

def route_after_judge(state, max_retries=settings.JUDGE_MAX_RETRIES):
    if state.get("judge_blocked"):
        return "blocked"
    if state.get("judge_passed"):
        return "output_guardrails"
    if state.get("retry_count", 0) >= max_retries:
        return "output_guardrails"
```

```
    return "assistant"

def route_after_output_guardrails(state,
batch_size=settings.LONG_TERM_MEMORY_BATCH_SIZE):
    if len(state["messages"]) % batch_size == 0:
        return "memory_extract"
    return "__end__"
```

Görev 7

```
retry_policy = RetryPolicy(max_attempts=settings.LLM_MAX_ATTEMPTS)

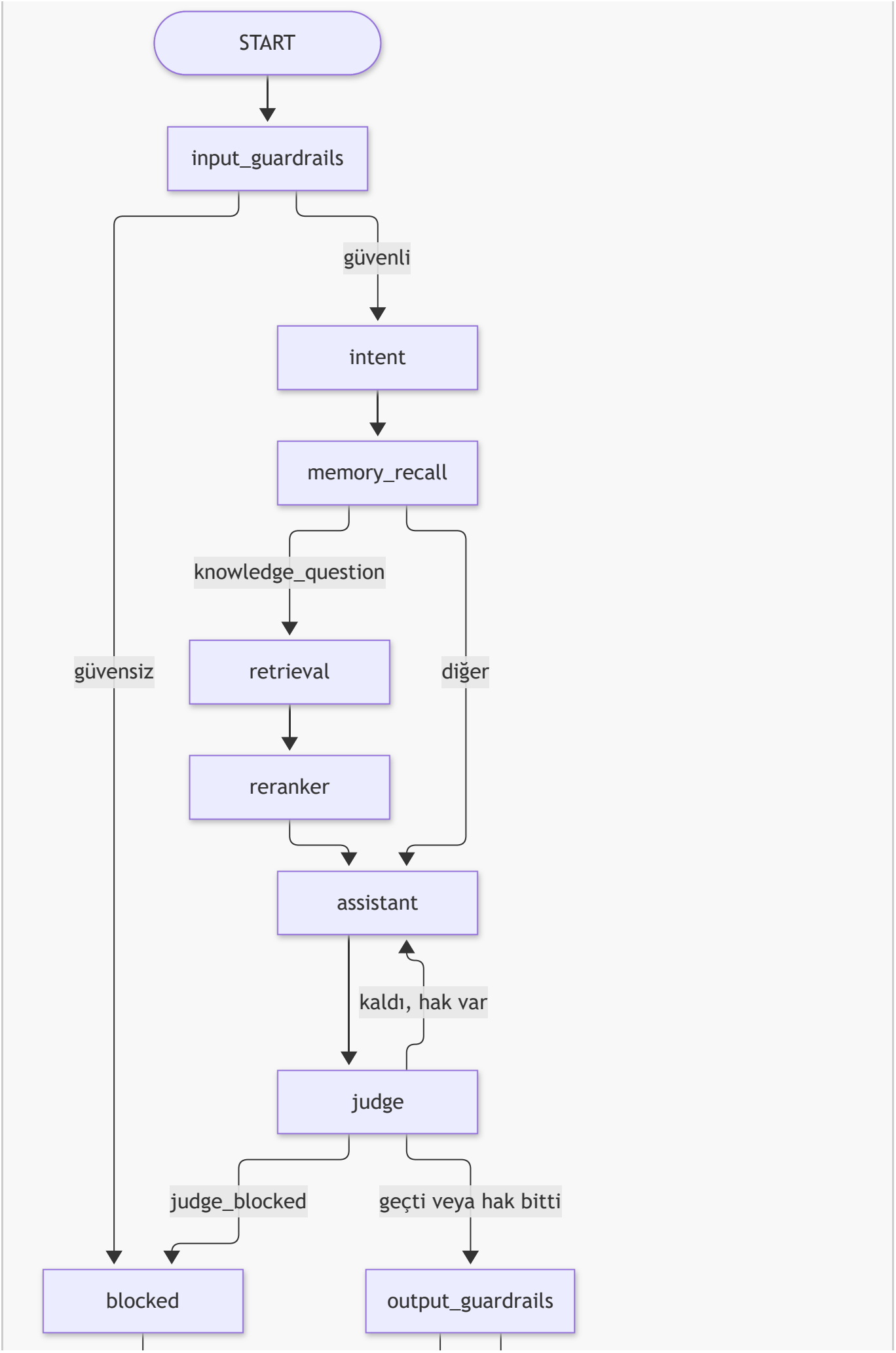
builder = StateGraph(AgentState, context_schema=Context)
builder.add_node("input_guardrails",
make_input_guardrails_node(guardrails_service))
builder.add_node("intent", make_intent_node(intent_chain),
retry_policy=retry_policy)
builder.add_node("retrieval", make_retrieval_node(retrieval_service),
retry_policy=retry_policy)
builder.add_node("reranker", make_reranker_node(reranker_chain),
retry_policy=retry_policy)
builder.add_node("assistant", make_assistant_node(assistant_chain),
retry_policy=retry_policy)
builder.add_node("judge", make_judge_node(judge_chain), retry_policy=retry_policy)
builder.add_node("output_guardrails",
make_output_guardrails_node(guardrails_service))
builder.add_node("memory_recall", make_memory_recall_node())
builder.add_node("memory_extract", make_memory_extract_node(memory_chain))
builder.add_node("blocked", blocked_response_node)

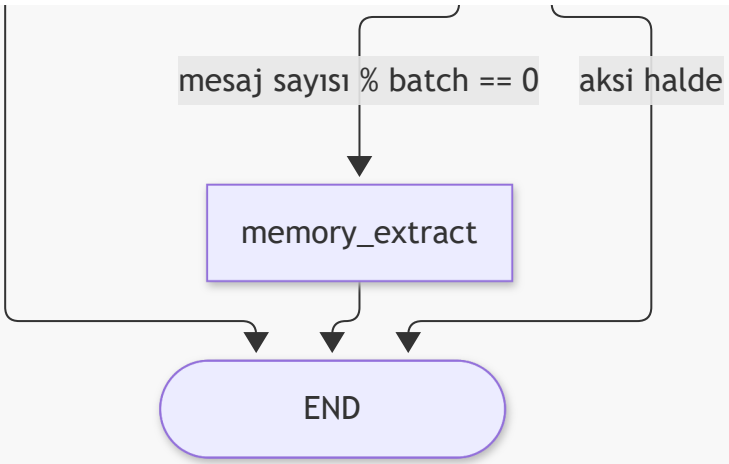
builder.add_edge(START, "input_guardrails")
builder.add_edge("intent", "memory_recall")
builder.add_edge("retrieval", "reranker")
builder.add_edge("reranker", "assistant")
builder.add_edge("assistant", "judge")
builder.add_edge("blocked", END)
builder.add_edge("memory_extract", END)

builder.add_conditional_edges("input_guardrails", route_after_input_guardrails)
builder.add_conditional_edges("memory_recall", route_after_memory_recall)
builder.add_conditional_edges("output_guardrails", route_after_output_guardrails)
builder.add_conditional_edges("judge", route_after_judge)

return builder.compile(checkpointer=checkpointer, store=store)
```

Dolu akış diyagramı:


START
input_guardrails
güvenli
güvensiz
intent
memory_recall
knowledge_question
retrieval
diğer
reranker
assistant
kaldı, hak var
judge
judge_blocked
geçti veya hak bitti
blocked
output_guardrails



Sık hatalar

- Judge router'ında sıranın karışması: engellenen yanıt yeniden denenir.
- `>` yerine `>=`: bir fazla deneme.
- Router'ı olan düğümden ayrıca `add_edge`: çift çıkış.
- Düğüm adı yazım hatası: derlemede "unknown node" hatası.
- LLM düğümlerine `retry_policy` verilmemesi: testler geçer ama geçici API hataları graph'ı düşürür; kod incelemesinde belirtin.

Bölüm 3: Tool ve Agent (Öğretmen)

Anlatım notu (10 dk)

- `tools/date_tool.py` üzerinden `@tool` ve `ToolRuntime` gösterin. Docstring'in LLM için kullanım kılavuzu olduğunu vurgulayın.
- Tahtada senaryo: asistan bir turda iki kez arar. `start` hesaplanmazsa asistanın "[1]" atfı hangi belgeyi gösterir?
- Agent döngüsü: model -> araç çağrısı -> araç sonucu -> model -> yapılandırılmış cevap. Middleware'lerin bu döngüdeki yerini gösterin.
- Kapanışta workflow ve agentic graph'ı aynı soruyla çalıştırıp loglarda farkı gösterin.

Görev 8

```
candidate_docs = await retrieval_service.search(query=query, top_k=candidates)
if not candidate_docs:
    return NOT_FOUND_ANSWER
try:
    documents = await rerank_documents(
        chain=reranker_chain,
        query=query,
        candidates=candidate_docs,
        top_k=top_k,
        min_score=min_score,
    )
except RerankerError as e:
    logger.warning(f"Rerank failed, falling back to vector order: {e}")
```

```
    documents = candidate_docs[:top_k]
if not documents:
    return NOT_FOUND_ANSWER
```

```
start = len(runtime.context.sources) + 1
runtime.context.sources.extend(documents)
formatted_sources = format_sources(documents, start=start)
return f"Bulunan kaynaklar:\n\n{formatted_sources}"
```

(Koddaki çözümde ek log satırları vardır; testler için gerekli değildir.)

Görev 9

```
llm = build_chat_model(model_name, temperature=temperature)
middleware_list = [
    ToolCallLimitMiddleware(tool_name=SEARCH_TOOL_NAME,
run_limit=settings.MAX_TOOL_CALLS),
    ToolCallLimitMiddleware(tool_name=DATE_TOOL_NAME,
run_limit=settings.MAX_TOOL_CALLS),
    ToolRetryMiddleware(
        max_retries=settings.TOOL_MAX_RETRIES,
        on_failure=_search_tool_failure_message,
    ),
]
return create_agent(
    model=llm,
    tools=[make_search_documents_tool(retrieval_service, reranker_chain),
get_today_date],
    response_format=ToolStrategy(AssistantResponse),
    context_schema=AgentContext,
    middleware=middleware_list,
)
```

Koddaki çözüm listeyi `cast(Sequence[Any], ...)` ile sarar; bu sadece tip denetleyicisi içindir.

Görev 10

```
messages = AGENT_ASSISTANT_TEMPLATE.format_messages(
    memories=_format_memories(state),
    intent=state["intent"],
    resolved_query=state.get("resolved_query") or last_user_msg.text,
    feedback=format_feedback(state.get("feedback")),
    history=history,
)
agent_context = AgentContext()
# ...
try:
```

```
        result = await agent.ainvoke({"messages": messages}, context=agent_context)
    except GraphRecursionError:
        ...  # hazır
    # ...
    return _assistant_update(response, agent_context.sources)
```

`agentic_graph`: `workflow_graph`'tan `retrieval` ve `reranker` düğümleri çıkarılır, asistan `make_agent_assistant_node(assistant_agent)` ile eklenir, `memory_recall -> assistant` sabit kenar olur. Koşullu kenarlar: `input_guardrails`, `judge`, `output_guardrails`.

Sık hatalar

- `start` sabit 1: `test_gorev8_ikinci_cagri_numaralandirmaya_devam_eder` kırmızı.
- Aday getirirken `top_k` kullanmak: `test_gorev8_aday_sayisi_candidates` kırmızı.
- `except Exception`: testler geçer ama programlama hataları yutulur; kod incelemesinde düzeltin.
- `AgentContext()` fabrikada bir kez: `test_gorev10_her_cagrida_yeni_context` kırmızı.
- `agent_context`'in `try` içinde oluşturulması: `GraphRecursionError` yolunda `UnboundLocalError`.
- `response_format=AssistantResponse` (ToolStrategy'siz): Gemini araçlarla birlikte yapılandırılmış çıktıyı reddedebilir.